UNIVERSIDAD PONTIFICIA ICAI ICADE COMILLAS MADRID

Sistemas de Almacenamiento NoSQL. HBase Arquitectura I

Máster en Big Data. Tecnología y Analítica Avanzada (MBD)

Escuela Técnica Superior de Ingeniería (ICAI)

comillas.edu

Índice

- Introducción e historia de HBase
- ¿Qué es HBase? y Comparativas
- ¿Por qué utilizar HBase?
- Fortalezas y Debilidades de HBase
- HBase en Producción
- Modelo de Datos y Diseño de Tablas
- Ejercicio Práctico y Discusión

comillas.edu

Introducción a HBase

- HBase es un sistema NoSQL orientado a columnas y basado en el modelo clave-valor que nos permite almacenar y recuperar grandes volúmenes de datos de forma rápida y escalable
- Es un sistema distribuido diseñado para alta escalabilidad y tolerancia a fallos.
- Se integra en el ecosistema Hadoop y aprovecha HDFS para tener almacenamiento distribuido.

https://hbase.apache.org/book.html

comillas.edu

Historia

- 2003 Google publica GFS y MapReduce
    - Fue creado para satisfacer la necesidad de Google de almacenar y gestionar petabytes de datos, asegurando **alta disponibilidad y escalabilidad**.
    - MapReduce, con su enfoque de dividir y paralelizar procesos, **revolucionó el análisis de datos**, permitiendo a Google procesar enormes volúmenes de información de manera eficiente.
- 2006 Google publica el artículo "*Bigtable: A Distributed Storage System for Structured Data*"
    - Sentó las bases conceptuales de HBase al combinar ideas de escalabilidad, tolerancia a fallos y un **modelo flexible basado en columnas y claves**.
    - La flexibilidad y el enfoque en el manejo de datos masivos de Bigtable sorprendieron a la industria, abriendo la puerta a **soluciones innovadoras** que rompieran con las limitaciones de los RDBMS tradicionales.
    - Expertos quedaron sorprendidos por la capacidad de Bigtable para escalar de manera horizontal y gestionar datos en un entorno distribuido
- *2007 Powerset crea HBase*
    - La filosofía de Bigtable inspiró a la comunidad open source y motivó la creación de HBase. Powerset adopta y adapta los conceptos de Bigtable para crear HBase en un entorno open source.

comillas.edu

Historia

- *2008 HBase subproyecto de Apache Hadoop*
    - La integración de HBase con Hadoop no solo amplió su adopción, sino que también permitió una sinergia entre el almacenamiento y el procesamiento de datos, creando soluciones integrales para Big Data.
- *2010 HBase se convierte en proyecto Top Level*
    - Se destacan las mejoras en la gestión de regiones, la integración con el ecosistema Hadoop y el aumento en la estabilidad y rendimiento.
    - Nivel de madurez y soporte comunitario suficiente.
- *2011 HBase 0.92 release*
    - Se optimizó la gestión de las regiones, facilitando su división y balanceo, mejorando la distribución de datos.
    - Se incluyeron los HFiles mejorando el rendimiento de L/E
- *2018 HBase 2.1.0*
    - Se optimizaron los mecanismos de replicación, permitiendo una mayor eficiencia en la sincronización de datos entre clústeres, además de reforzar la integración con sistemas de seguridad como Kerberos y mejorar el control de autorizaciones

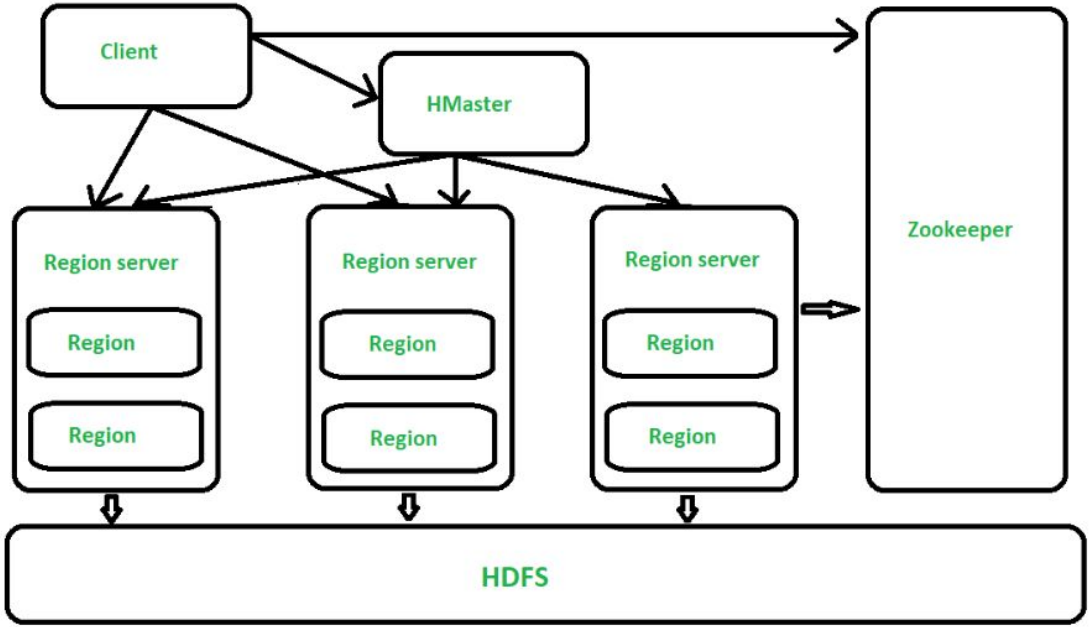
comillas.edu

¿Qué es HBase?

- HBase es la respuesta de la comunidad **open source** al problema de **almacenar y acceder** a volúmenes masivos de datos en tiempo real a diferencia de los sistemas tradicionales relacionales (RDBMS)
- HBase no requiere esquemas rígidos, permitiendo que se almacenen datos semi-estructurados que pueden cambiar en forma y estructura con el tiempo
- Basado en un **modelo clave-valor** y orientado a columnas.
- El modelo basado en **columnas** permite almacenar diferentes tipos de datos en una misma tabla.
- Destacar cómo HBase se **apoya en HDF**S para almacenar datos de forma distribuida. Esto significa que, al estar funcionando sobre Hadoop, se beneficia de la escalabilidad y la resiliencia de HDFS
- Capacidad de **escalar horizontalmente** añadiendo nodos al clúster.
- Desarrollado en **Java**, lo que favorece su integración con otras tecnologías de la plataforma Java.

comillas.edu

Arquitectura de HBase



comillas.edu

Arquitectura de HBase

- **HBase Master:** Coordina la asignación de regiones, gestiona operaciones administrativas y actúa como punto central de control.
- **RegionServers:** Responsables de manejar las operaciones de lectura y escritura.
    - Cada RegionServer administra varias regiones (porciones de la tabla) y se encarga de la distribución de datos en función de la row key.
    - Es crucial diseñar la row key de forma que los datos se distribuyan de manera uniforme entre las regiones
- **ZooKeeper:** Servicio de coordinación que gestiona el estado del clúster y asegura que todos los nodos estén sincronizados.
- **Peticiones de lectura/escritura:** una petición de escritura se recibe en el Master, se asigna a un RegionServer y luego se escribe en el clúster, utilizando HFiles y mecanismos de compactación para optimizar el rendimiento.
- Cuando se realiza una **operación de lectura**, el cliente se conecta al RegionServer que tiene la región correspondiente a la row key solicitada

comillas.edu

Características

- **Distribución y Balanceo Automático:**
  - Divide los datos en regiones distribuidas entre RegionServers, evitando cuellos de botella y asegurando un reparto uniforme de la carga de trabajo.
- **Recuperación automática** de servidores ante fallos
  - Implementa mecanismos de replicación y recuperación que garantizan la continuidad del servicio, incluso ante fallos inesperados.
- **Integración con Hadoop/HDFS**
  - Opera sobre HDFS, aprovechando la escalabilidad y resiliencia del sistema de ficheros distribuido.
- **Schemaless**
  - Permite almacenar datos semi-estructurados sin un esquema predefinido, facilitando la evolución del modelo de datos.
- **Lecturas y escrituras consistentes**
  - Garantiza consistencia a nivel de fila, asegurando que cada operación (lectura o escritura) se ejecute de forma aislada.
- **Soporte para Múltiples APIs** y Tecnologías:
  - Ofrece APIs en Java, REST, Avro, Thrift y compatibilidad con MapReduce, ampliando las posibilidades de integración y procesamiento paralelo."

comillas.edu

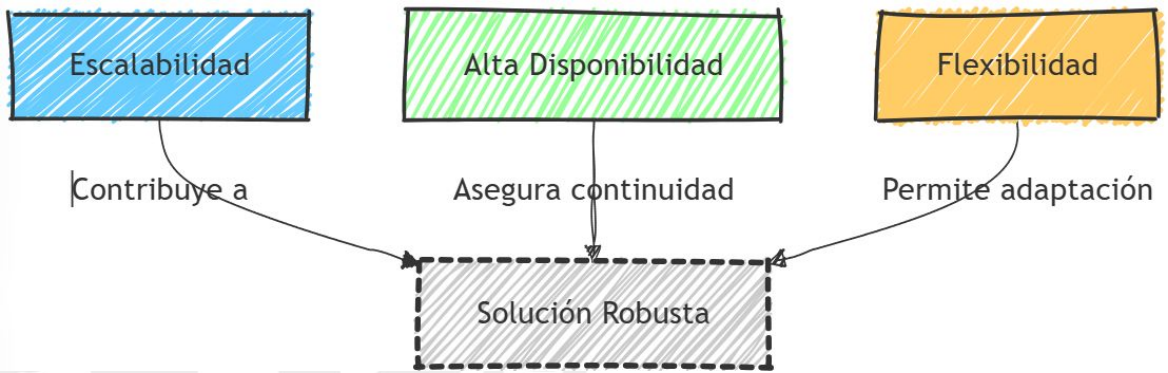
Cuándo utilizar HBase

- No es la solución para todos los problemas
- Primero, asegúrate de tener suficientes datos:
  - Si dispones de cientos de millones o miles de millones de filas, HBase es un buen candidato.
  - Si sólo tienes unas pocas miles/millones de filas, un RDBMS tradicional puede ser una mejor opción, ya que es posible que todos los datos terminen en uno o dos nodos y el resto del clúster permanezca inactivo
- Acepta la ausencia de características avanzadas de un RDBMS:
  - HBase no ofrece columnas tipadas, índices secundarios, transacciones o lenguajes de consulta avanzados.
  - Migrar de un RDBMS a HBase implica un rediseño completo de la aplicación, no simplemente cambiar el controlador JDBC.
- Asegúrate de contar con hardware suficiente

comillas.edu

Fortalezas de HBase

- Escalabilidad horizontal
- “Alta disponibilidad”, consistencia y tolerancia a fallos
- Integración con ecosistema Hadoop
- Flexibilidad del modelo schemaless
- Optimización de operaciones de lectura escritura



comillas.edu

Debilidades de HBase

- **Curva de Aprendizaje Elevada:**

  La configuración inicial y la administración de HBase requieren un conocimiento profundo, lo que puede dificultar su adopción para nuevos usuarios.

- **Complejidad en la Configuración:**

  La correcta configuración del clúster, la integración con ZooKeeper y la gestión de HDFS son aspectos críticos; errores en estos puntos pueden afectar gravemente el rendimiento.

- **Limitaciones en Operaciones Relacionales:**

  HBase no soporta operaciones de joins y consultas complejas propias de los RDBMS, lo que requiere rediseñar las aplicaciones para adaptarse a un modelo 'schemaless'.

- **Dependencia de la Infraestructura Subyacente:**

  Gran parte de la alta disponibilidad depende de HDFS y ZooKeeper, y en versiones tempranas (como la 0.92) las funcionalidades de replicación nativa eran limitadas.

- **Requerimientos de Hardware:**

  Para un funcionamiento óptimo, se necesita un clúster suficientemente grande (mínimo 3 DataNodes y un NameNode), lo que puede suponer un coste elevado.

comillas.edu

HBase vs HDFS

HDFS	HBase
Sistema de archivos distribuido, ideal para almacenar archivos muy grandes	Se construye sobre HDFS, aprovechando su escalabilidad y tolerancia a fallos.
Acceso completo al fichero	Acceso aleatorio a la información
Tamaño de bloque de 128 Mb	Bloque de 64 Kb
No es un sistema de archivos de propósito general; no optimiza búsquedas de registros individuales	Proporciona búsquedas y actualizaciones rápidas de registros en tablas grandes
Estructura estática	Estructura flexible
Paradigma escribe una vez / lee muchas	Optimizado para leer y escribir muchas veces
Se basa en la replicación (por defecto, 3 copias) para garantizar resiliencia y tolerancia a fallos.	Organiza internamente los datos en “StoreFiles” indexados para lograr consultas de alta velocidad.

comillas.edu

HBase vs RDBMS

HBase	RDBMS
Orientado a columnas	Orientado a filas
Esquema flexible, añadir columnas al vuelo	Esquema fijo
Millones de consultas por sg	Miles de consultas por sg
SQL	get/put/scan/etc*
Escala horizontalmente añadiendo HW	Difícil de fragmentar y escalar
Bueno con información estructurada y semi-estructurada	Bueno para información estructurada
No se insertan celdas vacías	Hay que insertar NULL para vacíos
UPSERT	INSERT y UPDATE
Indexación y ordenado sólo por clave	Indexación y ordenado por cualquier campo

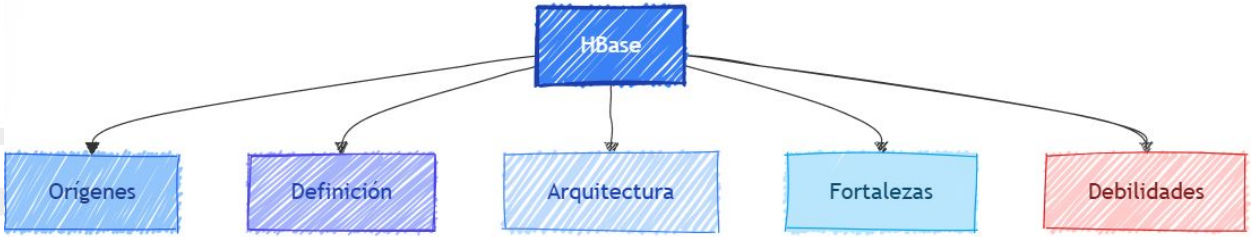
comillas.edu

HBase en Producción: Casos Reales

- Empresas como Facebook y Twitter utilizan HBase para gestionar petabytes de datos en tiempo real, garantizando baja latencia y alta concurrencia.
- HBase se utiliza en sistemas de mensajería, análisis de logs, detección de fraudes y aplicaciones IoT, donde la velocidad y la escalabilidad son esenciales.
- La replicación, el balanceo de carga y el monitoreo continuo permiten operar clústeres robustos y resilientes en entornos de alta demanda.

comillas.edu

Resumen



¿Cuál creéis que es el principal desafío al adoptar HBase en un entorno real y cómo se relaciona con su arquitectura y modelo de datos?

comillas.edu

De la teoría a la práctica: Modelo de datos

El **diseño de la tabla** es crucial para aprovechar la escalabilidad y flexibilidad de HBase

1) Impacto en el **Rendimiento**
   - Una buena estructura distribuye los datos de manera uniforme y minimiza ‘hot spots’.
2) Elección de la **Row Key**
   - Debe diseñarse para asegurar una distribución balanceada y facilitar búsquedas eficientes.
3) Definición de **Column Families**
   - Las familias se definen en la creación de la tabla y determinan cómo se almacenan físicamente los datos.
4) **Flexibilidad** del Modelo
   - El modelo 'schemaless' permite agregar nuevas columnas dinámicamente sin alterar la estructura existente.

Comprender y diseñar correctamente el modelo de datos es la base para desarrollar aplicaciones robustas en HBase.

comillas.edu

HBase Tables: Conceptos básicos

Término	Definición
Tabla	La información en HBase se almacena en tablas.
Fila	Cada fila se identifica por una Row Key única, la cual determina la distribución de los datos en el clúster y afecta directamente la eficiencia de las búsquedas
Familia de C.	Las columnas se agrupan en familias definidas en el momento de la creación de la tabla. Los datos de la misma familia se almacenan juntos físicamente, lo que mejora el rendimiento de lectura cuando se consulta un grupo específico de columnas.
Qualifier	Dentro de cada Column Family, los Qualifiers representan las columnas individuales. Los Qualifiers en HBase pueden variar de una fila a otra, lo que permite una gran flexibilidad para agregar nuevos atributos de forma dinámica.
Timestamp	Identifica la versión de la información lo que es crucial para historicos
Celda	Cada intersección que se produce entre la columna (FC + Qualifier) y fila

comillas.edu

Buenas Prácticas en el Diseño de Tablas

QUÉ HACER

Diseñar una Row Key Balanceada:
Utilizar patrones que distribuyan uniformemente la carga para evitar 'hot spots'.

Definir Column Families Coherentes:
Agrupar columnas que se consultan juntas para maximizar el rendimiento de lectura.

Planificar el Uso de Versionado:
Configurar adecuadamente el número de versiones a retener para evitar acumulación innecesaria de datos.

Modelar en Función de los Patrones de Consulta:
Analizar los requerimientos de las consultas para diseñar el esquema que minimice lecturas innecesarias.

QUÉ EVITAR

Sobrecargar Column Families:
No incluir demasiadas columnas en una misma familia, lo que puede degradar el rendimiento de lectura y escritura.

Diseñar Row Keys Predictibles:
Evitar patrones que puedan causar una distribución desigual de datos (ejemplo: claves secuenciales).

Ignorar el Impacto del Versionado:
No configurar el versionado de forma adecuada, lo que puede generar un crecimiento descontrolado del almacenamiento.

Rediseñar Sin Evaluar Patrones de Acceso:
No trasladar directamente diseños de RDBMS sin analizar las particularidades del modelo 'schemaless' de HBase

comillas.edu

Familias de columnas en HBase

Las familias de columnas son el pilar de la organización física de datos en HBase

- Se **definen** al crear la tabla y no pueden modificarse sin recrear la tabla.
- **Agrupan columnas** que habitualmente se consultan en conjunto, lo que mejora el rendimiento y permite que las operaciones de lectura sean más eficientes.
- Los datos dentro de una misma Column Family se **almacenan contiguamente** en disco. Una mala definición (por ejemplo, incluir demasiadas columnas diversas en la misma familia) puede provocar que se lean datos innecesarios, afectando el rendimiento.
- Un diseño adecuado de Column Families **optimiza** las lecturas y escrituras al minimizar accesos dispersos.

Ej: sistema de monitorización donde se agrupan los datos de “metadata” y “eventos” en familias de columnas separadas.

comillas.edu

Rows (Filas)

Las filas en HBase son identificadas de manera única por su Row Key y están compuestas por 1 o varias familias de columnas.

- **Identificación Única**: La Row Key es el identificador principal de cada registro, puede ser diseñada para optimizar patrones de acceso específicos.
- **Distribución de Datos:** La elección de la Row Key influye directamente en cómo se distribuye datos en el clúster (evitar 'hot spots').
- **Orden Lexicográfico:** Las filas se ordenan lexicográficamente, lo que facilita búsquedas y escaneos secuenciales.
- **Impacto en el Rendimiento:** Un diseño adecuado de la Row Key es esencial para lograr un balance de carga uniforme y acelerar las consultas.

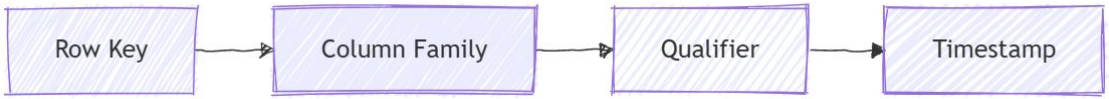
comillas.edu

Columnas y Qualifiers

En HBase, las **columnas** se definen de forma dinámica a través de los Qualifiers de un familia de columnas.

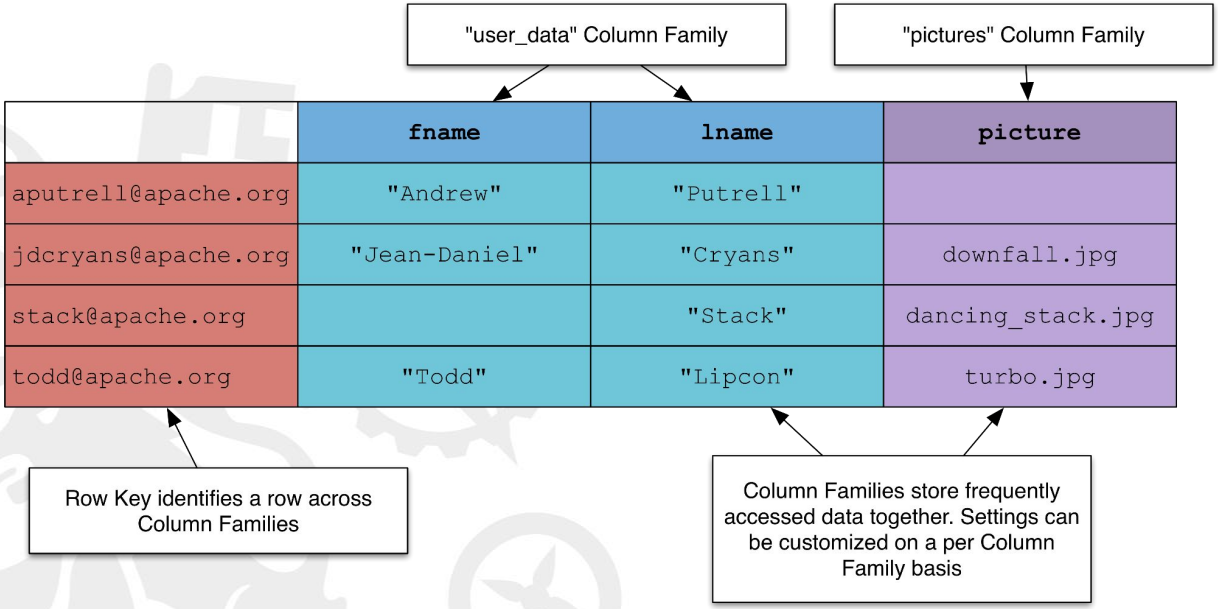
Qualifier

- Son los **nombres** de las columnas dentro de una familia de columnas.
- Pueden variar de una fila a otra, lo que aporta **gran flexibilidad**. Permite agregar nuevas columnas sin necesidad de alterar el esquema global.
- La combinación de Row Key, Column Family y Qualifier identifica de forma única cada **celda.** (unidad que contiene todas las versiones del dato)
- Permite **consultas precisas**: se pueden recuperar únicamente los datos necesarios, mejorando el rendimiento
- Cada celda puede almacenar **múltiples versiones** de un dato, identificadas por un timestamp.



comillas.edu

Almacenamiento fisico en HBase



comillas.edu

Almacenamiento físico en HBase

- **Datos como Arrays de Bytes:**
  HBase almacena todos los datos como arrays de bytes, lo que permite la flexibilidad para manejar cualquier tipo de información.

- **Organización en HFiles:**
  Los datos se escriben en HFiles, que son archivos estructurados y optimizados para lecturas y escrituras rápidas en HDFS.

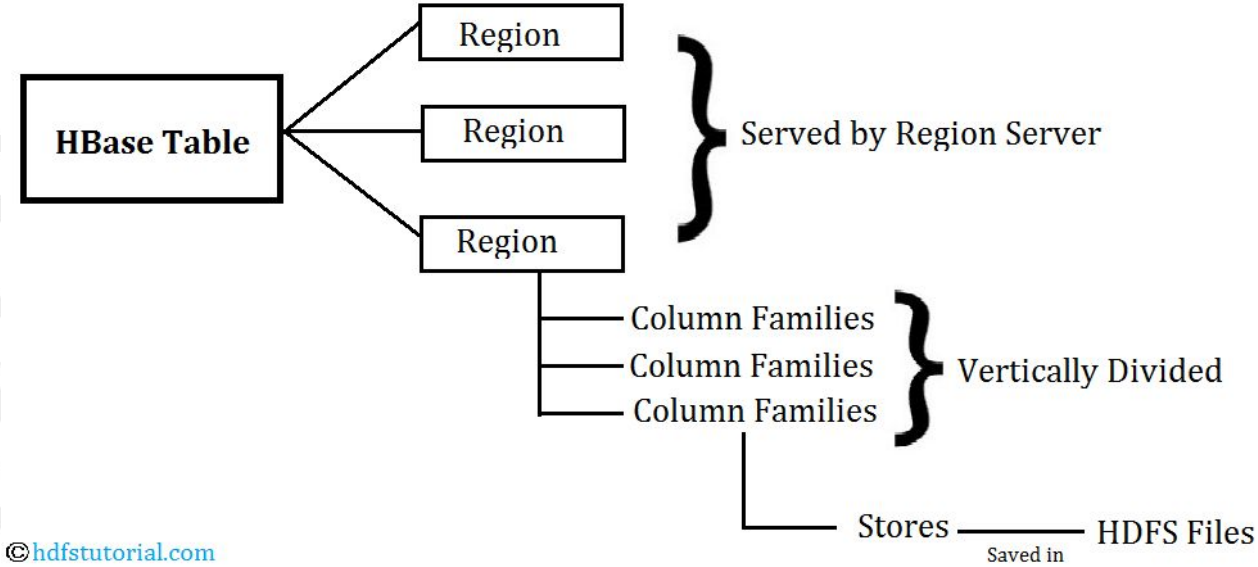
- **Agrupación por Column Family:**
  Dentro de cada HFile, los datos se organizan por Column Family, lo que garantiza que los datos relacionados se almacenen de forma contigua en disco.

- **Compresión y Optimización:**
  Se aplican técnicas de compresión y caching para mejorar el rendimiento y reducir el espacio en disco.

- **Celdas Vacías:**
  Las celdas sin datos no se almacenan físicamente, lo que optimiza el uso del espacio.

comillas.edu

Almacenamiento fisico en HBase



comillas.edu

Versionado físico

- **Concepto de Versionado:**
  Cada celda en HBase es la intersección de una Row Key y una columna (definida por la combinación de Column Family y Qualifier) y puede contener múltiples versiones del dato.

- **Identificación mediante Timestamps:**
  Cada versión está asociada a un timestamp, que indica el momento en que se escribió la versión, permitiendo acceder al historial de cambios.

- **Almacenamiento de Múltiples Versiones:**
  La celda lógica es el conjunto de todas las versiones almacenadas; esto es útil para auditorías y para recuperar estados anteriores de los datos.

- **Optimización del Acceso:**
  El sistema está diseñado para recuperar rápidamente la versión más reciente, pero también permite consultar versiones anteriores si es necesario.

comillas.edu

Versionado físico

Row key	Column	Timestamp	Valor
key1	cf1:Col1	1273746289103	*ValueA*
key1	cf1:Col2	1273878447049	*ValueB*
key2	cf1:Col1	1273516197868	*ValueF*
key2	cf1:Col2	1273871824184	*ValueG*
key3	cf1:Col1	1273616297446	*ValueK*
key3	cf1:Col2	1273971921442	*ValueL*

comillas.edu

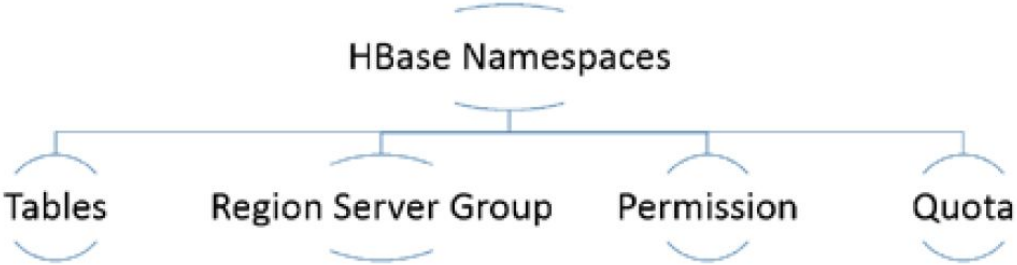
Prácticas guiadas Shell

HBase Namespace

HBase Tables

comillas.edu

Namespace



- Agrupación lógica de tablas
- Similar a un schema o database en RDBMS
- Namespaces predefinidos:
  - hbase – tablas internas
  - default – tablas sin namespace definido explícitamente

comillas.edu

HBase shell - DDL namespace

- Gestión de namespaces

```
create_namespace 'my_ns'

drop_namespace 'my_ns'
```

- Listar tablas existentes

```
list
```

- Describir tabla

```
describe 'my_ns.my_table'
```

comillas.edu

HBase shell - DDL create/alter

- Creación de tablas

```
create 'my_ns:my_table', 'cf1', 'cf2'

create 'my_ns:my_table', {NAME => 'cf1'},
{NAME => 'cf2', VERSIONS => 3}

alter 'my_ns:my_table', NAME => 'new_cf',
VERSIONS => 5
```

Propiedad	Posibles valores	Valor por defecto
COMPRESSION	*NONE, GZ, LZO, SNAPPY*	*NONE*
VERSIONS	*1+*	*1*
TTL	*1+*	*FOREVER*
BLOCKSIZE	*1 B – 2 GB*	*64 kB*
IN_MEMORY	*true,false*	*false*
BLOCKCACHE	*true,false*	*true*

comillas.edu

HBase shell - DDL enable/disable

- Deshabilitar/habilitar una tabla:

```
disable 'my_ns:my_table'
enable 'my_ns:my_table'
```

- Eliminar una tabla:

```
drop 'my_ns:my_table'
```

- Eliminar todos los registros de una tabla:

```
truncate 'my_ns:my_table'
```

comillas.edu

HBase shell - DML put

- Put: Ejecuta un *upsert* de la row. Por defecto se asocia un valor de timestamp correspondiente al momento actual (formato epoch). Si la family está configurada con varias versiones, se retienen los valores antiguos.

```
put 'my_ns:my_table', 'rowkey', 'cf:qual', 'value'
put 'my_ns:my_table', 'rowkey', 'cf:qual',
'value', <timestamp>
```

comillas.edu

HBase shell - DML get

- Get: Obtiene la row asociada a la *rowkey* especificada, es posible seleccionar las columnas a devolver:

```
get 'my_ns:my_table', 'rowkey'

get 'my_ns:my_table', 'rowkey', 'column'

get 'my_ns:my_table', 'rowkey', { COLUMN =>
['c1', 'c2'] }

get 'my_ns:my_table', 'rowkey', { COLUMN =>
'c1', VERSIONS => 2 }
```

comillas.edu

HBase shell - DML scan

- Scan: Permite obtener múltiples row, por defecto devuelve **todos** los datos existentes en la tabla. Es posible seleccionar las columnas a devolver y aplicar filtros de búsqueda.

```
scan 'my_ns:my_table'
scan 'my_ns:my_table', { LIMIT => 10 }
scan 'my_ns:my_table', { STARTROW => 'row1',
ENDROW => 'row2' }
scan 'my_ns:my_table', { LIMIT => 10 }
scan 'my_ns:my_table', { COLUMNS => ['c1',
'c2'] }
```

comillas.edu

HBase shell - DML delete

- Delete: Elimina una row, si se indica timestamp se eliminan esa versión y las anteriores. Inicialmente es un borrado lógico.

```
delete 'my_ns:my_table', 'rowkey', 'cf:qual'
delete 'my_ns:my_table', 'rowkey', 'cf:qual',
<timestamp>
deleteall 'my_ns:my_table', 'rowkey'
```

comillas.edu

HBase shell- DML count

- count: Realiza un conteo de todas las rows existentes en una tabla, es una operación costosa ya que realiza un barrido de todas las regiones de la tabla. Puede tardar minutos o incluso no terminar.

```
count 'my_ns:my_table'
count 'my_ns:my_table', { INTERVAL => 5000 }
count 'my_ns:my_table', { INTERVAL => 5000,
CACHE => 1000 }
```

comillas.edu

Ejercicio

Ejercicio 1.1: Practicando con la shell

comillas.edu